ilitoimistojärjestelmä

nish SaaS Accounting Office System — Full System Design

---

rsion: 1.0 — Initial Design

te: May 2026

ployment: SaaS Multi-tenant

thodology: Test-Driven Development (TDD)

debase Estimate: ~211,000 LOC (incl. tests)

ild Timeline: 18–24 months

Tilitoimistojärjestelmä — Architecture & Design Plan

Context

Building a SaaS-based Finnish accounting office system from scratch (greenfield repo). The platform serves accounting firms (tilitoimistot) as primary tenants — each firm subscribes and manages their own client companies (asiakkaat) as isolated sub-tenants. Accountants work on web; customers get a lightweight mobile+web portal. Must comply fully with Finnish accounting law, tax authority integrations, and GDPR.

User answers: - Deployment: SaaS (multiple tilitoimistot, platform-as-a-service) - Payroll: In-house Finnish engine (not 3rd party) - Backend stack: Architecture-driven recommendation (see below) - Mobile scope: Customer portal only (scan receipts, approve invoices, view reports) - Development approach: **Test-driven development (TDD)**

Tenancy Model

```
Platform (SaaS operator)
└── Tilitoimisto / Tenant  [1..N]
    └── Asiakas / Sub-tenant  [1..M per tilitoimisto]
```

Isolation: PostgreSQL Row Level Security (RLS) on every table. - `tenant_id` + `asiakas_id` columns everywhere - RLS policies: platform_admin sees all; tilitoimisto users see their asiakkaat; asiakas users see only their own data - RLS must be tested with a dedicated security test suite before any production deployment

Technology Stack

Layer	Technology	Justification
Backend API	Python 3.12 + FastAPI	Decimal precision for finance, strong typing (Pydantic), async I/O, best OCR/ML ecosystem
Database	PostgreSQL 16 + RLS	ACID, RLS for tenant isolation, JSONB for document metadata, pg_cron for scheduled jobs
Task queue	Celery + Redis	Async OCR, OmaVero submissions, bank sync, tulorekisteri
Web frontend	React 18 + TypeScript + Vite	Large ecosystem, shared component library between tilitoimisto and asiakas apps
Mobile	React Native + Expo	Customer portal only; camera, offline queue, push notifications
Object storage	S3-compatible (MinIO local, AWS S3 prod)	Tenant-scoped keys, immutable archive policy
Auth	Keycloak (self-hosted)	OIDC/OAuth2, MFA, Suomi.fi tunnistus integration
Infra	Kubernetes + Helm + Terraform	Multi-tenant scale, isolated namespaces per environment
CI/CD	GitHub Actions	Test gates before every merge
Monitoring	Prometheus + Grafana + Sentry	Financial system uptime and error tracking

Codebase Size Estimate (TDD ratio)

Layer	Production LOC	Test LOC	Total
Backend (Python)	~60,000	~55,000	~115,000
Web frontend (TS/React)	~40,000	~25,000	~65,000
Mobile (React Native)	~12,000	~8,000	~20,000
Infra / config / SQL	~8,000	~3,000	~11,000
Total	**~120,000**	**~91,000**	**~211,000**

TDD raises test LOC to ~75% of production code. Financial calculation modules (payroll, ALV) aim for near 100% coverage. This is a medium-large enterprise codebase — comparable to ~2-3 years of focused development by a 4-6 person team.

TDD Strategy

TDD is the right approach here because: - Payroll deduction errors = legal and financial liability - ALV rate errors = OmaVero penalties - Double-entry imbalance = corrupted books - Tenant isolation bugs = catastrophic data leak

Testing layers:

```
Unit tests          — pure financial calculations (payroll, ALV, double-entry balance
Integration tests   — API endpoints, DB queries, RLS enforcement
Contract tests      — OmaVero API, Tulorekisteri API, Finvoice XML schema
E2E tests           — full user flows (tilitoimisto onboards asiakas, posts tosite, s
Security tests      — dedicated RLS penetration suite (cross-tenant access attempts)
```

TDD rules for this project: - All financial calculation functions: write test with known Finnish tax scenario BEFORE implementation - ALV rates and payroll rates live in test fixtures, not hardcoded — tests break if rates not updated - Every OmaVero/Tulorekisteri integration has a mock

+ a contract test against sandbox API - RLS tests: automated suite attempts cross-tenant data access after every migration

---

Core Data Model

Tenancy tables

```
organizations          -- tilitoimistot (tenants)
  id, name, y_tunnus, subscription_plan, created_at

org_users              -- tilitoimisto staff
  id, org_id, user_id, role

customers              -- asiakkaat (sub-tenants)
  id, org_id, name, y_tunnus, tilikausi_alkaa, tilikausi_loppuu, alv_jakso

customer_users         -- asiakas portal users
  id, customer_id, user_id, role
```

Kirjanpito core

```
tilikartat             -- chart of accounts per asiakas
tilit                  -- accounts: numero, nimi, tyyppi, alv_koodi
tilikaudet             -- fiscal years: alkaa, loppuu, tila (avoin/suljettu/lukittu)
tositteet              -- vouchers: tyyppi, pvm, numero (auto), tila
viennit                -- journal lines: tili_id, debet, kredit, alv_%  (debet=kredit
```

Reskontra

```
myyntilaskut           -- AR invoices + Finvoice metadata
ostolaskut             -- AP invoices + approval state machine
suoritukset            -- payments matched to invoices
tilitapahtumat         -- bank statement lines (from CAMT.053/MT940)
```

Palkanlaskenta

```
tyontekijat            -- employees: henkilotunnus (encrypted), ennakonpidatys%, tyel
palkkakaudet           -- pay periods
palkat                 -- calculated pay: brutto, ennakonpidatys, tyel, sairausvakuut
palkka_verokortti      -- withholding tax card per employee per year
tulorekisteri_log      -- submission log + API responses
alv_vuosikorot         -- ALV rate table by year (NOT hardcoded)
palkkavakuutusmaksut   -- insurance rate table by year (NOT hardcoded)
```

ALV & Authorities

```
alv_kaudet             -- VAT periods per asiakas
alv_laskelmat          -- VAT calculation lines by code
omavero_lahetykset     -- OmaVero submission log
```

Archive & Compliance

```
asiakirjat             -- immutable document registry: s3_key, sha256, retention_unti
audit_log              -- append-only: user, action, before/after JSON, timestamp, ip
```

Module Breakdown

Module 1 — Auth & IAM

Roles: platform_admin, tilitoimisto_admin, kirjanpitaja, asiakas_admin, asiakas_user, tilintarkastaja - JWT + refresh tokens via Keycloak - RBAC enforced at API gateway + service level - MFA mandatory for all accountant roles - Suomi.fi tunnistus (Phase 2 option) - All auth events written to audit_log

Module 2 — Kirjanpito (Bookkeeping Core)

- Tilikartta from KILA standard template (customizable)
- Tilikausi state machine: avoin → suljettu → lukittu
- Double-entry enforced: `sum(debet) = sum(kredit)` per tosite (DB constraint)
- Tosite auto-numbering per kausi (YYYY + sequential)
- Period locking: no backdated posts to locked kausi
- Finnish ALV codes (1-99 series) on viennit

Module 3 — Myyntireskontra (AR)

- Myyntilasku with line items and ALV breakdown
- Finnish viitenumero generation (mod 97)
- Finvoice 3.0 XML + PEPPOL BIS 3.0 generation
- Auto-match incoming payments by viitenumero
- Asiakasrekisteri with YTJ lookup by Y-tunnus

Module 4 — Ostoreskontra (AP)

- Receive e-invoices via Finvoice operator (Maventa/Apix broker)
- OCR parse paper invoices (Tesseract)
- Approval state machine: `saapunut → odottaa_hyvaksyntaa → hyvaksytty → maksettu → kirjattu`
- SEPA PAIN.001 payment file generation
- Auto-post to kirjanpito after approval

Module 5 — ALV

- Rate table in DB per year (Sept 2024: standard changed 24% → 25.5%)
- Finnish ALV codes: 0%, 10%, 14%, 25.5% (standard), EU reverse charge, OSS
- Period calculator (kuukausittain / neljännesvuosittain based on liikevaihto)
- OmaVero API push with certificate-based OAuth2
- Correction submissions for late/amended periods

Module 6 — Palkanlaskenta (HIGHEST COMPLEXITY)

Finnish deductions (2024, all stored in rate tables): - Ennakonpidätys — from individual verokortti (fetched from OmaVero or entered manually) - TyEL työntekijä — 7.15% (<53yo), 8.65% (53-62yo) - Sairausvakuutusmaksu — 1.16% (employee) - Työttömyysvakuutusmaksu — 0.79% (employee) - Employer: TyEL, sairausvakuutus, tapaturma, ryhmähenkivakuutus

Process flow: 1. Define palkkauskausi → enter työtunnit/kuukausipalkat 2. Auto-calculate all deductions from rate tables 3. Generate palkkalaskelma PDF 4. Auto-post palkkakirjanpito viennit 5. Submit to Tulorekisteri API (within 5 days of payment day) 6. Generate TyEL ilmoitus to pension company 7. SEPA PAIN.001 for salary disbursement

Rate tables are DB records, not code constants — annual update is an admin workflow.

Module 7 — Tilinpäätös (Year-End Closing)

- Guided checklist workflow with blocking steps
- Tuloslaskelma: kululajipohjainen (default) + toimintokohtainen
- Tase: Finnish standard format
- Liitetiedot templates: mikroyritys / pk-yritys / suuri yritys
- XBRL/iXBRL export (mandatory for large companies from 2024)
- Auto-closing entries (depreciation, accruals)
- Tilikausi locked after tilinpäätös approval — irreversible

Module 8 — Viranomaisintegraatiot

System	Protocol	Purpose
OmaVero	REST + OAuth2 + org cert	ALV, ennakkovero, yhteisöilmoitus
Tulorekisteri	REST + cert	Real-time palkka ilmoitukset
YTJ	REST	Y-tunnus lookup, company data
Finvoice/ PEPPOL	SFTP/AS4 via operator	E-invoice send/receive

All external API calls: retry logic, timeout, circuit breaker, full response logging to `omavero_lahetykset` / `tulorekisteri_log`.

Module 9 — Pankkiintegraatiot

- **Inbound:** CAMT.053 XML + MT940 import → tilitapahtumat table
- **Auto-reconciliation:** match by viitenumero, then by amount+date+toimittaja
- **Outbound:** SEPA PAIN.001 generated from approved ostolaskut + palkat
- Supported banks: Nordea, OP, Danske, Handelsbanken, S-Pankki (all use CAMT.053 now)

Module 10 — Asiakasportaali

Web + Mobile (React Native/Expo): - Dashboard: velat, saatavat, ALV-status, next deadline - Receipt upload: drag-drop (web) / camera scan (mobile) - Invoice approval queue with comment field - Report viewer: P&L, tase, ALV summary (read-only) - Threaded messaging per document with kirjanpitäjä - Offline receipt queue on mobile (sync on reconnect) - Push notifications (Firebase): "Lasku X odottaa hyväksyntääsi"

Module 11 — Tilitoimiston Master View

- All clients dashboard with RAG status (red/amber/green)

- Deadline tracker: ALV, palkat, tilinpäätös — all clients unified calendar
- Missing receipts count per asiakas
- Kirjanpitäjä workload assignment and view
- Alert engine: "3 clients have ALV due in 5 days"

Module 12 — Raportointi & Arkisto

- Standard reports: tuloslaskelma, tase, pääkirja, päiväkirja, ALV-erittely, reskontra, palkkayhteenveto
- Export: PDF (WeasyPrint), Excel (openpyxl), CSV
- 10-year immutable archive (S3 object lock, WORM policy)
- SHA256 hash stored per document at upload — integrity verifiable
- GDPR: henkilötunnus encrypted at rest with customer-specific key; right-to-erasure handled by key deletion

Module 13 — OCR & Auto-posting

- Pipeline: upload → S3 → Celery worker → Tesseract OCR → extract (toimittaja, pvm, summa, ALV, viite)
- ML tili suggestion: trained on asiakas's own posting history
- Kirjanpitäjä reviews suggestion, accepts or corrects
- Active learning: corrections fed back to model

Module 14 — Notification & Deadline Engine

- pg_cron + Celery beat for scheduled checks
- Deadlines tracked: ALV (12th of following month), Tulorekisteri (5 days post-payment), Tilinpäätös (4 months post-tilikausi)
- Channels: email (SendGrid), push (Firebase), in-app

API Design

- REST (not GraphQL) — financial resources have clear semantics
- OpenAPI 3.1 spec — contract-first, generate clients
- Versioned: `/api/v1/...`

- JWT bearer tokens with `tenant_id`, `asiakas_id`, `role` claims
- Separate API contexts: `/tilitoimisto/...` (accountant) vs `/portal/...` (customer)

Phased Build Roadmap

Phase	Duration	Deliverables
1 — Foundation	3-4 mo	Infra, auth/IAM, tilikartta, kirjanpito core, tilikausi, audit log
2 — Reskontra	2-3 mo	AR, AP, approval workflow, bank import, reconciliation, PAIN.001
3 — ALV & Authorities	2 mo	ALV engine, OmaVero API, YTJ, Finvoice operator
4 — Payroll	3-4 mo	Full palkanlaskenta, Tulorekisteri API, TyEL, palkkakirjanpito
5 — Year-End	2 mo	Tilinpäätös workflow, XBRL, statutory reports, PDF/Excel
6 — Portal & Mobile	2-3 mo	Asiakasportaali web, React Native app, OCR pipeline, master dashboard
7 — Intelligence	ongoing	ML auto-posting, advanced analytics, Suomi.fi tunnistus

Phases 1-3 = usable product. Full system ~18-24 months.

Key Design Decisions (must resolve before coding)

1. **Finvoice operator:** Use Maventa or Apix as broker — do not implement operator directly

2. **OmaVero cert:** Software cert per tenant stored in secrets manager (HSM for production)
3. **ALV + payroll rates:** Never hardcoded — DB rate tables with annual admin update workflow. Tests will fail if rates are stale.
4. **Tilikausi lock:** Enforced at DB level (trigger), not just application layer
5. **Henkilötunnus:** Encrypted at rest, never in logs, access restricted to payroll module only
6. **Mobile:** Customer portal only — kirjanpitäjät on web exclusively
7. **Tilikartta:** KILA standard as seed template, customer can extend account numbering but cannot break standard group ranges

---

Complexity & Risk Assessment

Module	Complexity	Risk	Key Mitigation
Auth & RLS	Medium	**Critical**	Dedicated security test suite; pentest before launch
Kirjanpito core	Medium	Medium	DB constraint for double-entry; TDD on all posting logic
Myyntireskontra	Medium	Low	Standard; Finvoice schema has XSD validation
Ostoreskontra	Medium	Low	Approval state machine tested exhaustively
ALV engine	High	**High**	TDD with known Verohallinto scenarios; rate table tests
Palkanlaskenta	**Very High**	**High**	TDD first; dedicated team; rate tables tested annually
Tulorekisteri	High	High	Real-time mandate; test against sandbox API; retry + idempotency
OmaVero integration	High	High	Certificate management; sandbox testing; response logging
Bank integrations	High	Medium	Format coverage tests with real bank sample files
Tilinpäätös / XBRL	High	Medium	XBRL taxonomy well-documented; validate against Eurofiling
OCR / ML	High	Low	ML accuracy improves over time; human review always available
Multi-tenancy RLS	High	**Critical**	Automated cross-tenant penetration tests on every migration

Module	Complexity	Risk	Key Mitigation
GDPR vs retention	Medium	High	Legal review needed — right-to-erasure vs. 10-year accounting law conflict

Verification Plan

- **Unit:** All financial functions (ALV calc, payroll deductions, double-entry balance) — pytest with parameterized Finnish tax scenarios from Verohallinto published examples
- **Integration:** API endpoint tests (httpx + pytest) with real PostgreSQL including RLS
- **Contract:** OmaVero sandbox API + Tulorekisteri test environment — validate XML schemas
- **Security:** Automated suite: login as asiakas_user of Tenant A, attempt to read Tenant B data — must return 403/empty at every endpoint
- **E2E:** Playwright (web) + Detox (mobile) — core flows: onboard asiakas, post tosite, submit ALV ilmoitus, run payroll, view tilinpäätös
- **Compliance:** Annual review against Kirjanpitolaki updates and Verohallinto guideline changes

Public API Integration Reference

All APIs verified against current public documentation. Sandbox environments confirmed available for all critical integrations.

1. Verohallinto / Vero API

Purpose: ALV ilmoitus, ennakkovero, verokortti fetch, ennakonpidätys rates

Property	Detail
Auth	mTLS client certificate (PKCS#12) + `vero-softwarekey` header (software-specific key)
Base URL	`api-developer.vero.fi` (sandbox), production URL via vero.fi registration
Format	JSON (primary), XML for some interfaces
Encoding	UTF-8
Sandbox hours	08:00–16:00 weekdays only (maintenance outside hours)
Cost	Free
SDK	None official — direct REST calls; contact `ohjelmistotalot@vero.fi` for dev support

Key operations: - `POST` ALV-ilmoitus (VAT return submission) - `GET` verokortti (employee withholding tax card) — fetched per henkilötunnus - `POST` ennakkovero payments - `GET` ennakonpidätystiedot (withholding tax info for employer) - `POST` yhteisöilmoitus (annual corporate tax return)

Implementation notes: - One certificate per tilitoimisto tenant (stored in secrets manager) - Certificate issued by Verohallinto after registration — ~2-4 week lead time - All requests logged to `omavero_lahetykset` with full response body - Circuit breaker: if sandbox is outside 08:00–16:00, mock responses in tests

---

2. Tulorekisteri (Incomes Register) API

Purpose: Real-time palkka ilmoitukset (mandatory within 5 days of payment)

Property	Detail
Auth	X.509 certificate-based (separate certs for: earnings submission, benefits submission, data retrieval)
Submission methods	Real-time web service (per-report), Deferred web service (batch), SFTP
Format	**XML only** — no BOM character allowed, UTF-8
Sandbox	Stakeholder testing environment available
SDK	None official — direct calls; major payroll software (Palkkaus.fi) as reference

Key operations: - Submit palkka ilmoitus (wage report) per employee per payment - Submit corrections to previous reports - Cancel submitted reports - Retrieve submitted data (data distribution interface)

Implementation notes: - 3 separate certificates: earnings, benefits, retrieval — all must be provisioned at onboarding - Idempotency critical: duplicate submissions must be detected and rejected - 5-day deadline is calendar days from payment date — deadline engine must track - Test against stakeholder environment before any production payroll run - XML schema validation must happen client-side before submission

---

3. YTJ / PRH Open Data API

Purpose: Company lookup by Y-tunnus, auto-populate asiakas details, validate business existence

Property	Detail
Auth	**None required** — fully public API
Base URL	`https://avoindata.prh.fi/opendata-ytj-api/v3`
Spec	OpenAPI 3.0.1
Format	JSON
Rate limit	300 requests/minute (shared across all users)
Sandbox	No separate sandbox — test directly against live API (data is public register)
Community SDKs	`jannewaren/ytj_client` (Python), `nikked/Avoindata.fi-Python-Tools`

Key operations: - `GET /companies/{businessId}` — fetch by Y-tunnus - `GET /companies?name=...` — search by name - Daily bulk JSON export available for local caching

Implementation notes: - Cache Y-tunnus lookups locally (data updates daily, no need to hit API per request) - Does NOT include sole proprietorships (toiminimet) — manual entry fallback needed - No email/phone numbers — only register data - Use for KYC: validate that asiakas y_tunnus is a real active business

---

4. Finvoice 3.0 (Format Specification)

Purpose: Finnish national e-invoicing XML format — mandatory for bank network; standard for B2B

Property	Detail
Type	XML format specification (not an API — delivered via operators)
Maintained by	Finance Finland (Finanssiala ry)
Schema	XSD + Schematron validation rules — publicly downloadable
Encoding	UTF-8 or ISO 8859-15
Latest version	3.0 (implemented Sept 2025+)
EU compliance	Mapped to EN 16931 standard
Spec URL	`finanssiala.fi/en/topics/finvoice-standard/`

Implementation notes: - Generate Finvoice 3.0 XML in-house — spec is publicly available - Validate against XSD schema before sending to operator - Amounts: support 2-5 decimal places - Delivery: always via operator (Maventa or Apix) — never direct to bank - TEAPPSXML 3.0 is the equivalent operator-to-operator routing format (fully compatible)

---

5. PEPPOL BIS 3.0

Purpose: EU-wide e-invoicing — required for public sector (B2G) in Finland

Property	Detail
Type	UBL 2.1 XML format over PEPPOL network
Access via	Certified PEPPOL access point (Maventa is access point in Finland)
Documentation	`docs.peppol.eu/poacc/billing/3.0/`
Validators	`invoice-navigator.eu/validator` (free, public)
Sandbox	Via Maventa sandbox

Implementation notes: - Use Maventa as both Finvoice operator AND PEPPOL access point (single integration) - Validate UBL 2.1 XML against Schematron rules before sending - Growing adoption: mandatory for public sector procurement; B2B use increasing

---

6. Maventa (Primary E-invoice Operator)

Purpose: Send/receive Finvoice + PEPPOL invoices; operator network routing

Property	Detail
Auth	OAuth2 — `client_id` (Company UUID) + `client_secret` (API key) + `vendor_api_key` (partner key)
Token endpoint	`POST /oauth2/token`
REST API docs	`documentation.maventa.com/rest-api/`
Swagger	`swagger.maventa.com`
Sandbox	Full sandbox available — requires registration
Scale	100,000+ companies in Finland

Key AutoXChange (AX) API operations: - Create company on Maventa network - `POST` send invoice (Finvoice 3.0 / TEAPPSXML / PEPPOL BIS) - `GET` receive invoices from network - Register webhooks for delivery status - Activate networks (PEPPOL, Scan, bank network)

Implementation notes: - One Maventa account per tilitoimisto tenant OR white-label partner account (vendor_api_key model) - Partner/vendor integration: single API key per software product, sub-accounts per customer - Pricing: Netstamps (prepaid credits) or monthly invoice model - Webhook registration for invoice delivery status — store in `finvoice_lahetykset` log table

---

7. Apix Messaging (Secondary / Fallback Operator)

Purpose: Alternative e-invoice operator; fallback if Maventa unavailable

Property	Detail
Auth	SHA-256 HMAC digest — `TransferID` + `TransferKey`
Method	`PUT` to submit documents
Format	ZIP file containing XML data files + PDF attachments
API docs	`wiki.apix.fi/rest-api-for-external-usage`
Pricing	Transaction-based (Netstamps), no fixed costs
Sandbox	Available

Implementation notes: - Different auth model (SHA-256 digest, not OAuth2) — wrap in dedicated client class - Supports automatic format conversion for recipients - Plan: Maventa as primary, Apix as fallback / for specific customers

---

8. Bank Open Banking APIs (PSD2)

Purpose: Account information (tilitapahtumat), payment initiation (PAIN.001 alternative)

Nordea

Property	Detail
Auth	OAuth2 + SCA (Nordea ID app)
Portal	`developer.nordeaopenbanking.com`
Sandbox	Full sandbox, free access
SDK	C#, Java, Python samples available
Scale	6,500+ sandbox users, 500+ live companies

OP Pankki (largest in Finland)

Property	Detail
Auth	OAuth2 + SCA
Portal	`op-developer.fi`
Sandbox	Available, open to all developers
Integrators	GoCardless, Nordigen, Yapily, Klarna also aggregate OP data

Danske Bank Finland

Property	Detail
Auth	OAuth2 + SCA
Portal	`developers.danskebank.com`
Sandbox	**Restricted in Finland** — requires TPP license or Open Banking UK directory membership

Implementation notes: - Prefer CAMT.053 file import for most customers (simpler, no OAuth2 per-customer SCA) - Open Banking API

for customers who want real-time bank feed (advanced tier feature) - Consider aggregator (Nordigen/GoCardless) to handle multi-bank SCA complexity - S-Pankki and Handelsbanken: CAMT.053 file upload only (no public PSD2 API)

---

9. SEPA ISO 20022 Bank Formats

Purpose: Payment file (PAIN.001) and bank statement (CAMT.053) processing

Format	Direction	Purpose
`pain.001.001.03`	Outbound	Customer credit transfer initiation (salary + invoice payments)
`camt.053.001.02`	Inbound	Bank-to-customer account statement
`camt.052.001.02`	Inbound	Intraday transaction report (optional, real-time)

Implementation notes: - CAMT.053: primary bank statement format for all Finnish banks - MT940: legacy format — still supported by some banks, must handle both - PAIN.001: generate per payment batch — group by bank, one file per run - Nordea supports both standard and extended camt.053 — handle both variants - Validation: validate XML against ISO 20022 schema before sending to bank - Finance Finland ISO 20022 Payments Guide 2024 is the implementation reference

---

API Summary Table

API	Type	Auth	Format	Sandbox	Priority
Verohallinto Vero API	Government	mTLS cert + key	JSON	Yes (08-16 only)	P0
Tulorekisteri	Government	X.509 cert (3x)	XML only	Yes	P0
YTJ / PRH	Government	None	JSON	Live only (free)	P1
Finvoice 3.0	Format spec	Via operator	XML	Via operator	P0
Maventa AX API	Operator	OAuth2	JSON/ XML	Yes	P0
Apix	Operator	SHA-256 HMAC	ZIP+XML	Yes	P1 (fallback)
PEPPOL BIS 3.0	Format spec	Via access point	UBL XML	Via Maventa	P1
Nordea PSD2	Bank	OAuth2 + SCA	JSON	Yes	P2
OP Bank PSD2	Bank	OAuth2 + SCA	JSON	Yes	P2
Danske Bank PSD2	Bank	OAuth2 + SCA	JSON	Restricted (FI)	P2
CAMT.053 / PAIN.001	Bank format	Bank file transfer	XML	Bank dep.	P0

P0 = required for MVP | **P1** = required Phase 3 | **P2** = advanced tier feature

---

Integration Architecture Pattern

All external API clients follow the same structure:

```
ExternalAPIClient
  ├── authenticate()       — cert load / OAuth2 token fetch + refresh
  ├── submit(payload)      — send with retry (exp. backoff, max 3 attempts)
  ├── validate(response)   — assert schema compliance
  └── log(request, response) → external_api_log table
```

Separate log table per integration: `omavero_log`, `tulorekisteri_log`, `finvoice_log`, `bank_log` All logs: immutable, append-only, retained 10 years (same as kirjanpito aineisto).

Critical Files to Create (Phase 1)

```
backend/
  app/
    core/           -- config, database, auth, RLS middleware
    modules/
      kirjanpito/   -- tositteet, viennit, tilikartta, tilikausi
      alv/          -- rate tables, period calc, OmaVero client
      palkanlaskenta/ -- deduction engine, tulorekisteri client
      reskontra/    -- AR, AP, reconciliation
      tilinpaatos/  -- year-end workflow, XBRL
    tests/
      unit/
      integration/
      security/     -- RLS cross-tenant tests

frontend/
  apps/
    tilitoimisto/   -- master view, kirjanpito UI
    portal/         -- customer web portal

mobile/             -- React Native / Expo (customer portal)

infra/
  k8s/
  terraform/

docs/
  api/              -- OpenAPI 3.1 spec (contract-first)
```